## ASSESSMENT HANDOUT

## WEB PROGRAMMING

| | |
|---|---|
| **MODULE CODE** | CCS2210 |
| **MODULE TITLE** | Web Programming |
| **PROGRAMME** | BSc (Hons) in Computer Science |
| **DEPARTMENT** | Computer Science |
| **CREDITS** | 10 |
| **STAGE OF STUDY** | 2 |
| **SEMESTER/SESSION** | Fall 2025/26 |
| **LOCATION** | Athens |
| **STAFF** | Kosmas Kosmopoulos |
| **E-MAIL** | kkosmopoulos@athtech.gr |
| **STAFF OFFICE** | 3 Sofias Sliman Street, Athens |

| | |
|---|---|
| **ASSESSMENT NUMBER** | 1 |
| **CONTRIBUTION** | 70% of final mark |
| **ASSESSMENT TITLE** | Dynamic Web App |
| **ASSESSMENT TYPE** | Groupwork |
| **HAND-OUT DATE** | Week 4 |
| **SUBMISSION DATE** | Week 12 |
| **FEEDBACK DATE** | |

### LEARNING OUTCOMES

Upon completion of this piece of assessment, a student will be able to:

- use HTML5's semantic and syntactical components within a webpage
- use client-side JavaScript to add interactivity and dynamic content to a webpage
- use modern server-side frameworks to implement the logic for a database-driven server-side web application
- use CSS to provide styling to the components of a webpage
- realize the architecture and manage the components required for dynamic web applications
- design, implement and evaluate a professional, dynamic, database-driven web application

### ASSESSMENT CRITERIA

The web application as a whole will be assessed according to the following criteria:

- **UI / UX design [10%]**
  - Usability (via my heuristic evaluation of your UI)
  - Navigation / pagination
  - User support to tasks
  - Error handling
- **Performance / Security [20%]**
  - Sessions
  - Form validation
  - Browser compatibility
  - Unauthorised access
- **Functional requirements [20%]**
  - Login requirement
  - Form requirement
  - Search requirement
  - Shorting results
- **Front-end development quality [20%]**
  - Proper use of Fundamental technologies (HTML5, CSS3 and JS)
- **Back-end development quality [30%]**
  - Database design and implementation
  - Proper use of Node.js and Express framework
  - Rest API implementation

**DETAILED DESCRIPTION**

In this coursework assignment you will be working in groups of 4 to 5 students, to create a web application. The context of the web application is left to you (each team) but it must be formally approved by me A meeting will be scheduled in-class within 1 after this handout is made available to you in order to discuss and finalize the context and topic of your web application. All members of the group should be present. I strongly suggest that you have at least three alternative ideas that fit the application requirements (see next section).

# 1. Web Application Requirements

### 1.1 Users

Your system must involve users logging into the system with a username and password. The email of the user must also be provided. Note that forms for creating a user account and forms for logging into the system <u>do not count</u> towards any further requirements below.

### 1.2 Forms and user input

There must be at least one form. The forms should be of some complexity so as a general guideline, the following recommendation is given:

- At least three input fields.
- At least one set of radio buttons with at least four options.
- At least one set of check boxes with at least four options.
- At least one text area.

In addition, you must allow uploading of file(s). A maximum file size restriction should be set, as well as, restriction on the types of files that can be uploaded.

Note that all the above are not restrictive; it depends on the context of your web application.

### 1.3 Preserving Continuity after Login

Sessions must be used in order to preserve certain data across subsequent accesses. How long a session lasts depends on the context of your application.

### 1.4 The Database

The database of your application should not be a complicated (e.g. like the ones you did for the practical in the databases; those involved many tables and many relations). However, it cannot be a simple database (e.g. one or two tables/models). The minimum number of tables/models for the database of your system is three and there should be a minimum two connections (relations) between tables in the database. As an example, consider the classic 'customer' makes an 'order' for an 'item' scenario. This example has three tables (customer, order, item) and two relations (customer-order, order-item). The database should be populated with adequate and valid data.

### 1.5 Operations to/from the Database

Your system must do the following operations to a database at least once:

- read existing data from the database
- write new data to the database
- update existing data to the database
- search the database (discussed below)

In addition, your web application must provide "Search" functionality. Note that the search text box does not count towards the forms described in section 1.2. The search functionality must search the contents of the database and display results properly. When your web application displays the results to the users (after a query), you must provide the following functionality:

- **Sorting:** normally results will be returned in a tabular format with rows and columns. When clicking on the column title (e.g. this could be the attribute name), the results should be dynamically sorted by that attribute.
- **Pagination**: at least for one query, the results must be more than ten rows. Your web application must only display the first 10 results and then offer "next ten results" functionality properly.

### 1.6 Context

Your system must have a context and identity. In other words, you are not allowed to create sample database tables, sample HTML forms and sample interfaces, JavaScript and CSS code in order to achieve the functionality described above. The web application that you will create must have a clear purpose and a specific context. This part demonstrates your creativity.

## 2. Technical Requirements

The project that your team will create must adhere to the following non-negotiable technical requirements:

### 2.1 Front and back end requirements

For the front-end:

- It must use HTML5 (with semantic and syntactic tags).
- It must utilize Cascading Style Sheets (CSS3).
- It must utilize JavaScript with DOM manipulation.
- Extra credit will be given for using bootstrap.

For the back-end:

- It must utilize Node.js with Express for the back-end.
- A restful API should be used.
- It must utilize either the MySQL relational database management system, or MongoDB with Mongoose.
- Must use EJS

You cannot use a content management system. You are also required to deliver your system using Docker.

### 2.2 User Interface Issues, Error Handling, Security and Performance

User friendliness, aesthetics, navigation, handling of errors (e.g. malformed user input), security and performance of the web application are very important. Emphasis should be placed on all of these issues!

## 3. Meetings and video demonstration

Except for the initial meeting to discuss the topic of your application, each team is required to have two formal progress meetings with the lecturer during the semester. These meetings will take place on weeks 7 and 10 during class. Failure to attend (without a valid reason) any of the meetings will negatively affect your personal engagement mark assigned by the lecturer.

In addition, each team will have to prepare a video of 10 to 15 minutes long, where:

a) The web application functionalities will be demonstrated, in order to support the application requirements as discussed in section 1.

b) Each member of the group will explicitly discuss their contribution to the team effort (whether it is to code, design etc.). Failure of a team member to present their work will result in that team member getting a 0 to the personal engagement mark assigned by the lecturer.

It is the team's leader's responsibility to produce the video, with the assistance of all the team members.

## 4. Peer Assessment

You are also requested to individually complete the peer assessment form at the end of the project. The individual grade of each group member will be equally determined by the peer evaluation and the lecturer via the demonstration and comments in the peer evaluation that each student will make regarding their own contribution.

| **SUBMISSION** |
|---|
| Each team is required to submit the following in digital form:<br>● All the code for the implemented system to a GitHub repo.<br>● A 10 to 15 minutes screencast video demonstrating and <u>evaluating</u> the web application. I expect that each team member will present in the video the parts of the application that they were involved in. Submit at the classroom.<br>Each member of a team is required to submit in digital form:<br>● The peer assessment form at Teammates<br>**Zero Tolerance Submission Policy**: In this piece of assessment, **no late submissions are accepted.** |

| USE OF GENERATIVE ARTIFICIAL INTELLIGENCE | |
|---|---|
| AI | You will have the opportunity to use GenAI on many occasions during your studies. Based on the learning outcomes of this module, in this assessment you are allowed the limited and acknowledged use of GenAI (for creating the content of the website specify specific tasks that fall under the marking criteria). You must declare/cite any use of GenAI tools according to the given specific guidelines. Suspected unacknowledged or false declaration of GenAI use may still be investigated under the Use of Unfair Means Policy as stated in the Regulations. |

| NOTE |
|---|
| *This piece of assessment should be completed and submitted by the student (or group of students in group work) without assistance from or communication with another person either external or fellow student (outside the group) or any AI type of content generator. Students should understand that not working on their own will be considered grounds for unfair means and will result in a fail mark for this work and might invoke disciplinary actions. This piece of assessment will be continuously checked for its academic integrity until student's graduation and the mark will be revised if it is found to breach the unfair means policy. It is at the instructor's discretion to conduct an oral examination which will result in the award of the final grade for that particular piece of work.* |